JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class System

java.lang.Object
    java.lang.System

```
public final class System
extends Object
```

The System class contains several useful class fields and methods. It cannot be instantiated.

Among the facilities provided by the System class are standard input, standard output, and error output streams; access to externally defined properties and environment variables; a means of loading files and libraries; and a utility method for quickly copying a portion of an array.

Since:

JDK1.0

Field Summary

Fields

Modifier and Type	Field	Description
static PrintStream	err	The "standard" error output stream.
static InputStream	in	The "standard" input stream.
static PrintStream	out	The "standard" output stream.

Method Summary

Deprecated Methods

Modifier and Type	Method	Description
static void	**arraycopy**(**Object** src, int srcPos, **Object** dest, int destPos, int length)	Copies an array from the specified source array, beginning at the specified position, to the specified position of the destination array.
static **String**	**clearProperty**(**String** key)	Removes the system property indicated by the specified key.
static **Console**	**console**()	Returns the unique **Console** object associated with the current Java virtual machine, if any.
static long	**currentTimeMillis**()	Returns the current time in milliseconds.
static void	**exit**(int status)	Terminates the currently running Java Virtual Machine.
static void	**gc**()	

		Runs the garbage collector.
static Map<String,String>	getenv()	Returns an unmodifiable string map view of the current system environment.
static String	getenv(String name)	Gets the value of the specified environment variable.
static Properties	getProperties()	Determines the current system properties.
static String	getProperty(String key)	Gets the system property indicated by the specified key.
static String	getProperty(String key, String def)	Gets the system property indicated by the specified key.
static SecurityManager	getSecurityManager()	Gets the system security interface.
static int	identityHashCode(Object x)	Returns the same hash code for the given object as would be returned by the default method

		hashCode(), whether or not the given object's class overrides hashCode().
static **Channel**	**inheritedChannel**()	Returns the channel inherited from the entity that created this Java virtual machine.
static **String**	**lineSeparator**()	Returns the system-dependent line separator string.
static void	**load**(**String** filename)	Loads the native library specified by the filename argument.
static void	**loadLibrary**(**String** libname)	Loads the native library specified by the `libname` argument.
static **String**	**mapLibraryName**(**String** libname)	Maps a library name into a platform-specific string representing a native library.
static long	**nanoTime**()	Returns the current value of the running Java Virtual Machine's high-resolution time source, in nanoseconds.

static void	**runFinalization**()	Runs the finalization methods of any objects pending finalization.
static void	**runFinalizersOnExit**(boolean value)	**Deprecated.** This method was originally designed to enable or disable running finalizers on exit. Running finalizers on exit was disabled by default. If enabled, then the finalizers of all objects whose finalizers had not yet been automatically invoked were to be run before the Java runtime exits. That behavior is inherently unsafe. It may result in finalizers being called on live objects while other threads are concurrently manipulating those objects,

		resulting in erratic behavior or deadlock.
static void	**setErr**(**PrintStream** err)	Reassigns the "standard" error output stream.
static void	**setIn**(**InputStream** in)	Reassigns the "standard" input stream.
static void	**setOut**(**PrintStream** out)	Reassigns the "standard" output stream.
static void	**setProperties**(**Properties** props)	Sets the system properties to the `Properties` argument.
static **String**	**setProperty**(**String** key, **String** value)	Sets the system property indicated by the specified key.
static void	**setSecurityManager**(**SecurityManager** s)	Sets the System security.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

in

```
public static final InputStream in
```

The "standard" input stream. This stream is already open and ready to supply input data. Typically this stream corresponds to keyboard input or another input source specified by the host environment or user.

out

```
public static final PrintStream out
```

The "standard" output stream. This stream is already open and ready to accept output data. Typically this stream corresponds to display output or another output destination specified by the host environment or user.

For simple stand-alone Java applications, a typical way to write a line of output data is:

```
System.out.println(data)
```

See the `println` methods in class `PrintStream`.

See Also:

`PrintStream.println()`, `PrintStream.println(boolean)`, `PrintStream.println(char)`, `PrintStream.println(char[])`, `PrintStream.println(double)`, `PrintStream.println(float)`, `PrintStream.println(int)`, `PrintStream.println(long)`, `PrintStream.println(java.lang.Object)`, `PrintStream.println(java.lang.String)`

err

```
public static final PrintStream err
```

The "standard" error output stream. This stream is already open and ready to accept output data.

Typically this stream corresponds to display output or another output destination specified by the host environment or user. By convention, this output stream is used to display error messages or other information that should come to the immediate attention of a user even if the principal output stream, the value of the variable `out`, has been redirected to a file or other destination that is typically not continuously monitored.

Method Detail

setIn

```
public static void setIn(InputStream in)
```

Reassigns the "standard" input stream.

First, if there is a security manager, its `checkPermission` method is called with a `RuntimePermission("setIO")` permission to see if it's ok to reassign the "standard" input stream.

Parameters:

`in - the new standard input stream.`

Throws:

`SecurityException - if a security manager exists and its checkPermission method doesn't allow reassigning of the standard input stream.`

Since:

`JDK1.1`

See Also:

`SecurityManager.checkPermission(java.security.Permission)`, `RuntimePermission`

setOut

```
public static void setOut(PrintStream out)
```

Reassigns the "standard" output stream.

First, if there is a security manager, its `checkPermission` method is called with a `RuntimePermission("setIO")` permission to see if it's ok to reassign the "standard" output stream.

Parameters:

`out - the new standard output stream`

Throws:

`SecurityException - if a security manager exists and its checkPermission method doesn't allow reassigning of the standard output stream.`

Since:

`JDK1.1`

See Also:

SecurityManager.checkPermission(java.security.Permission), RuntimePermission

setErr

```
public static void setErr(PrintStream err)
```

Reassigns the "standard" error output stream.

First, if there is a security manager, its checkPermission method is called with a RuntimePermission("setIO") permission to see if it's ok to reassign the "standard" error output stream.

Parameters:

err - the new standard error output stream.

Throws:

SecurityException - if a security manager exists and its checkPermission method doesn't allow reassigning of the standard error output stream.

Since:

JDK1.1

See Also:

SecurityManager.checkPermission(java.security.Permission), RuntimePermission

console

```
public static Console console()
```

Returns the unique Console object associated with the current Java virtual machine, if any.

Returns:

The system console, if any, otherwise null.

Since:

1.6

inheritedChannel

```
public static Channel inheritedChannel()
                                 throws IOException
```

Returns the channel inherited from the entity that created this Java virtual machine.

This method returns the channel obtained by invoking the `inheritedChannel` method of the system-wide default `SelectorProvider` object.

In addition to the network-oriented channels described in `inheritedChannel`, this method may return other kinds of channels in the future.

Returns:

The inherited channel, if any, otherwise null.

Throws:

`IOException` - If an I/O error occurs

`SecurityException` - If a security manager is present and it does not permit access to the channel.

Since:

1.5

setSecurityManager

```
public static void setSecurityManager(SecurityManager s)
```

Sets the System security.

If there is a security manager already installed, this method first calls the security manager's `checkPermission` method with a `RuntimePermission("setSecurityManager")` permission to ensure it's ok to replace the existing security manager. This may result in throwing a `SecurityException`.

Otherwise, the argument is established as the current security manager. If the argument is `null` and no security manager has been established, then no action is taken and the method simply returns.

Parameters:

s - the security manager.

Throws:

`SecurityException` - if the security manager has already been set and its checkPermission method doesn't allow it to be replaced.

See Also:

`getSecurityManager()`, `SecurityManager.checkPermission(java.security.Permission)`, `RuntimePermission`

getSecurityManager

```
public static SecurityManager getSecurityManager()
```

Gets the system security interface.

Returns:

if a security manager has already been established for the current application, then that security manager is returned; otherwise, null is returned.

See Also:

setSecurityManager(java.lang.SecurityManager)

currentTimeMillis

```
public static long currentTimeMillis()
```

Returns the current time in milliseconds. Note that while the unit of time of the return value is a millisecond, the granularity of the value depends on the underlying operating system and may be larger. For example, many operating systems measure time in units of tens of milliseconds.

See the description of the class `Date` for a discussion of slight discrepancies that may arise between "computer time" and coordinated universal time (UTC).

Returns:

the difference, measured in milliseconds, between the current time and midnight, January 1, 1970 UTC.

See Also:

Date

nanoTime

```
public static long nanoTime()
```

Returns the current value of the running Java Virtual Machine's high-resolution time source, in nanoseconds.

This method can only be used to measure elapsed time and is not related to any other notion of system or wall-clock time. The value returned represents nanoseconds since some fixed but arbitrary *origin* time (perhaps in the future, so values may be negative). The same origin is used by all invocations of this method in an instance of a Java virtual machine; other virtual machine instances are likely to use a different origin.

This method provides nanosecond precision, but not necessarily nanosecond resolution (that is, how frequently the value changes) - no guarantees are made except that the resolution is at least as good as that of `currentTimeMillis()`.

Differences in successive calls that span greater than approximately 292 years (2^{63} nanoseconds) will not correctly compute elapsed time due to numerical overflow.

The values returned by this method become meaningful only when the difference between two such values, obtained within the same instance of a Java virtual machine, is computed.

For example, to measure how long some code takes to execute:

```
long startTime = System.nanoTime();
// ... the code being measured ...
long estimatedTime = System.nanoTime() - startTime;
```

To compare two nanoTime values

```
long t0 = System.nanoTime();
...
long t1 = System.nanoTime();
```

one should use `t1 - t0 < 0`, not `t1 < t0`, because of the possibility of numerical overflow.

Returns:

`the current value of the running Java Virtual Machine's high-resolution time source, in nanoseconds`

Since:

`1.5`

arraycopy

```
public static void arraycopy(Object src,
                             int srcPos,
                             Object dest,
                             int destPos,
                             int length)
```

Copies an array from the specified source array, beginning at the specified position, to the specified position of the destination array. A subsequence of array components are copied from the source array referenced by `src` to the destination array referenced by `dest`. The number of components copied is equal to the `length` argument. The components at positions `srcPos` through

`srcPos+length-1` in the source array are copied into positions `destPos` through `destPos+length-1`, respectively, of the destination array.

If the `src` and `dest` arguments refer to the same array object, then the copying is performed as if the components at positions `srcPos` through `srcPos+length-1` were first copied to a temporary array with `length` components and then the contents of the temporary array were copied into positions `destPos` through `destPos+length-1` of the destination array.

If `dest` is `null`, then a `NullPointerException` is thrown.

If `src` is `null`, then a `NullPointerException` is thrown and the destination array is not modified.

Otherwise, if any of the following is true, an `ArrayStoreException` is thrown and the destination is not modified:

- The `src` argument refers to an object that is not an array.
- The `dest` argument refers to an object that is not an array.
- The `src` argument and `dest` argument refer to arrays whose component types are different primitive types.
- The `src` argument refers to an array with a primitive component type and the `dest` argument refers to an array with a reference component type.
- The `src` argument refers to an array with a reference component type and the `dest` argument refers to an array with a primitive component type.

Otherwise, if any of the following is true, an `IndexOutOfBoundsException` is thrown and the destination is not modified:

- The `srcPos` argument is negative.
- The `destPos` argument is negative.
- The `length` argument is negative.
- `srcPos+length` is greater than `src.length`, the length of the source array.
- `destPos+length` is greater than `dest.length`, the length of the destination array.

Otherwise, if any actual component of the source array from position `srcPos` through `srcPos+length-1` cannot be converted to the component type of the destination array by assignment conversion, an `ArrayStoreException` is thrown. In this case, let ***k*** be the smallest nonnegative integer less than length such that `src[srcPos+`*k*`]` cannot be converted to the component type of the destination array; when the exception is thrown, source array components from positions `srcPos` through `srcPos+`*k*`-1` will already have been copied to destination array positions `destPos` through `destPos+`*k*`-1` and no other positions of the destination array will have been modified. (Because of the restrictions already itemized, this paragraph effectively applies only to the situation where both arrays have component types that are reference types.)

Parameters:

src - the source array.

srcPos - starting position in the source array.

dest - the destination array.

destPos - starting position in the destination data.

length - the number of array elements to be copied.

Throws:

IndexOutOfBoundsException - if copying would cause access of data outside array bounds.

ArrayStoreException - if an element in the src array could not be stored into the dest array because of a type mismatch.

NullPointerException - if either src or dest is null.

identityHashCode

```
public static int identityHashCode(Object x)
```

Returns the same hash code for the given object as would be returned by the default method hashCode(), whether or not the given object's class overrides hashCode(). The hash code for the null reference is zero.

Parameters:

x - object for which the hashCode is to be calculated

Returns:

the hashCode

Since:

JDK1.1

getProperties

```
public static Properties getProperties()
```

Determines the current system properties.

First, if there is a security manager, its checkPropertiesAccess method is called with no arguments. This may result in a security exception.

The current set of system properties for use by the getProperty(String) method is returned as a Properties object. If there is no current set of system properties, a set of system properties is first created and initialized. This set of system properties always includes values for the following keys:

| Key |

	Description of Associated Value
`java.version`	Java Runtime Environment version
`java.vendor`	Java Runtime Environment vendor
`java.vendor.url`	Java vendor URL
`java.home`	Java installation directory
`java.vm.specification.version`	Java Virtual Machine specification version
`java.specification.maintenance.version`	Java Runtime Environment specification maintenance version, may be interpreted as a positive integer *(optional, see below)*
`java.vm.specification.vendor`	Java Virtual Machine specification vendor
`java.vm.specification.name`	Java Virtual Machine specification name
`java.vm.version`	Java Virtual Machine implementation version
`java.vm.vendor`	Java Virtual Machine implementation vendor
`java.vm.name`	Java Virtual Machine implementation name
`java.specification.version`	Java Runtime Environment specification version
`java.specification.vendor`	Java Runtime Environment specification vendor
`java.specification.name`	Java Runtime Environment specification name
`java.class.version`	Java class format version number
`java.class.path`	Java class path
`java.library.path`	List of paths to search when loading libraries
`java.io.tmpdir`	Default temp file path
`java.compiler`	Name of JIT compiler to use
`java.ext.dirs`	Path of extension directory or directories **Deprecated.** *This property, and the mechanism which implements it, may be removed in a future release.*
`os.name`	Operating system name

os.arch	Operating system architecture
os.version	Operating system version
file.separator	File separator ("/" on UNIX)
path.separator	Path separator (":" on UNIX)
line.separator	Line separator ("\n" on UNIX)
user.name	User's account name
user.home	User's home directory
user.dir	User's current working directory

The java.specification.maintenance.version property is defined if the specification implemented by this runtime at the time of its construction had undergone a maintenance release. When defined, its value identifies that maintenance release. To indicate the first maintenance release this property will have the value "1", to indicate the second maintenance release this property will have the value "2", and so on.

Multiple paths in a system property value are separated by the path separator character of the platform.

Note that even if the security manager does not permit the getProperties operation, it may choose to permit the getProperty(String) operation.

Returns:

the system properties

Throws:

SecurityException - if a security manager exists and its checkPropertiesAccess method doesn't allow access to the system properties.

See Also:

setProperties(java.util.Properties), SecurityException, SecurityManager.checkPropertiesAccess(), Properties

lineSeparator

```
public static String lineSeparator()
```

Returns the system-dependent line separator string. It always returns the same value - the initial value of the system property line.separator.

On UNIX systems, it returns "\n"; on Microsoft Windows systems it returns "\r\n".

Returns:

the system-dependent line separator string

Since:

1.7

setProperties

```
public static void setProperties(Properties props)
```

Sets the system properties to the `Properties` argument.

First, if there is a security manager, its `checkPropertiesAccess` method is called with no arguments. This may result in a security exception.

The argument becomes the current set of system properties for use by the `getProperty(String)` method. If the argument is `null`, then the current set of system properties is forgotten.

Parameters:

`props - the new system properties.`

Throws:

`SecurityException - if a security manager exists and its checkPropertiesAccess method doesn't allow access to the system properties.`

See Also:

`getProperties(), Properties, SecurityException, SecurityManager.checkPropertiesAccess()`

getProperty

```
public static String getProperty(String key)
```

Gets the system property indicated by the specified key.

First, if there is a security manager, its `checkPropertyAccess` method is called with the key as its argument. This may result in a SecurityException.

If there is no current set of system properties, a set of system properties is first created and initialized in the same manner as for the `getProperties` method.

Parameters:

`key - the name of the system property.`

Returns:

`the string value of the system property, or null if there is no property with that key.`

Throws:

SecurityException - if a security manager exists and its checkPropertyAccess method doesn't allow access to the specified system property.

NullPointerException - if key is null.

IllegalArgumentException - if key is empty.

See Also:

setProperty(java.lang.String, java.lang.String), SecurityException, SecurityManager.checkPropertyAccess(java.lang.String), getProperties()

getProperty

```
public static String getProperty(String key,
                                 String def)
```

Gets the system property indicated by the specified key.

First, if there is a security manager, its checkPropertyAccess method is called with the key as its argument.

If there is no current set of system properties, a set of system properties is first created and initialized in the same manner as for the getProperties method.

Parameters:

key - the name of the system property.

def - a default value.

Returns:

the string value of the system property, or the default value if there is no property with that key.

Throws:

SecurityException - if a security manager exists and its checkPropertyAccess method doesn't allow access to the specified system property.

NullPointerException - if key is null.

IllegalArgumentException - if key is empty.

See Also:

setProperty(java.lang.String, java.lang.String), SecurityManager.checkPropertyAccess(java.lang.String), getProperties()

setProperty

```
public static String setProperty(String key,
                                 String value)
```

Sets the system property indicated by the specified key.

First, if a security manager exists, its `SecurityManager.checkPermission` method is called with a `PropertyPermission(key, "write")` permission. This may result in a SecurityException being thrown. If no exception is thrown, the specified property is set to the given value.

Parameters:

`key - the name of the system property.`

`value - the value of the system property.`

Returns:

`the previous value of the system property, or null if it did not have one.`

Throws:

`SecurityException - if a security manager exists and its checkPermission method doesn't allow setting of the specified property.`

`NullPointerException - if key or value is null.`

`IllegalArgumentException - if key is empty.`

Since:

`1.2`

See Also:

`getProperty(java.lang.String)`, `getProperty(java.lang.String)`, `getProperty(java.lang.String, java.lang.String)`, `PropertyPermission`, `SecurityManager.checkPermission(java.security.Permission)`

clearProperty

```
public static String clearProperty(String key)
```

Removes the system property indicated by the specified key.

First, if a security manager exists, its `SecurityManager.checkPermission` method is called with a `PropertyPermission(key, "write")` permission. This may result in a SecurityException being thrown. If no exception is thrown, the specified property is removed.

Parameters:

key - the name of the system property to be removed.

Returns:

the previous string value of the system property, or null if there was no property with that key.

Throws:

SecurityException - if a security manager exists and its checkPropertyAccess method doesn't allow access to the specified system property.

NullPointerException - if key is null.

IllegalArgumentException - if key is empty.

Since:

1.5

See Also:

getProperty(java.lang.String), setProperty(java.lang.String, java.lang.String), Properties, SecurityException, SecurityManager.checkPropertiesAccess()

getenv

public static String getenv(String name)

Gets the value of the specified environment variable. An environment variable is a system-dependent external named value.

If a security manager exists, its checkPermission method is called with a RuntimePermission("getenv."+name) permission. This may result in a SecurityException being thrown. If no exception is thrown the value of the variable name is returned.

System properties and *environment variables* are both conceptually mappings between names and values. Both mechanisms can be used to pass user-defined information to a Java process. Environment variables have a more global effect, because they are visible to all descendants of the process which defines them, not just the immediate Java subprocess. They can have subtly different semantics, such as case insensitivity, on different operating systems. For these reasons, environment variables are more likely to have unintended side effects. It is best to use system properties where possible. Environment variables should be used when a global effect is desired, or when
an external system interface requires an environment variable (such as PATH).

On UNIX systems the alphabetic case of name is typically significant, while on Microsoft Windows systems it is typically not. For example, the expression

`System.getenv("FOO").equals(System.getenv("foo"))` is likely to be true on Microsoft Windows.

Parameters:

`name - the name of the environment variable`

Returns:

`the string value of the variable, or null if the variable is not defined in the system environment`

Throws:

`NullPointerException - if name is null`

`SecurityException - if a security manager exists and its checkPermission method doesn't allow access to the environment variable name`

See Also:

`getenv(), ProcessBuilder.environment()`

getenv

```
public static Map<String,String> getenv()
```

Returns an unmodifiable string map view of the current system environment. The environment is a system-dependent mapping from names to values which is passed from parent to child processes.

If the system does not support environment variables, an empty map is returned.

The returned map will never contain null keys or values. Attempting to query the presence of a null key or value will throw a `NullPointerException`. Attempting to query the presence of a key or value which is not of type `String` will throw a `ClassCastException`.

The returned map and its collection views may not obey the general contract of the `Object.equals(java.lang.Object)` and `Object.hashCode()` methods.

The returned map is typically case-sensitive on all platforms.

If a security manager exists, its `checkPermission` method is called with a `RuntimePermission("getenv.*")` permission. This may result in a `SecurityException` being thrown.

When passing information to a Java subprocess, system properties are generally preferred over environment variables.

Returns:

`the environment as a map of variable names to values`

Throws:

`SecurityException` - if a security manager exists and its `checkPermission` method doesn't allow access to the process environment

Since:

1.5

See Also:

`getenv(String)`, `ProcessBuilder.environment()`

exit

```
public static void exit(int status)
```

Terminates the currently running Java Virtual Machine. The argument serves as a status code; by convention, a nonzero status code indicates abnormal termination.

This method calls the `exit` method in class `Runtime`. This method never returns normally.

The call `System.exit(n)` is effectively equivalent to the call:

```
Runtime.getRuntime().exit(n)
```

Parameters:

`status` - exit status.

Throws:

`SecurityException` - if a security manager exists and its `checkExit` method doesn't allow exit with the specified status.

See Also:

`Runtime.exit(int)`

gc

```
public static void gc()
```

Runs the garbage collector.

Calling the `gc` method suggests that the Java Virtual Machine expend effort toward recycling unused objects in order to make the memory they currently occupy available for quick reuse. When control returns from the method call, the Java Virtual Machine has made a best effort to reclaim space from all discarded objects.

The call `System.gc()` is effectively equivalent to the call:

```
Runtime.getRuntime().gc()
```

See Also:

`Runtime.gc()`

runFinalization

```
public static void runFinalization()
```

Runs the finalization methods of any objects pending finalization.

Calling this method suggests that the Java Virtual Machine expend effort toward running the `finalize` methods of objects that have been found to be discarded but whose `finalize` methods have not yet been run. When control returns from the method call, the Java Virtual Machine has made a best effort to complete all outstanding finalizations.

The call `System.runFinalization()` is effectively equivalent to the call:

```
Runtime.getRuntime().runFinalization()
```

See Also:

`Runtime.runFinalization()`

runFinalizersOnExit

```
@Deprecated
public static void runFinalizersOnExit(boolean value)
```

Deprecated. *This method was originally designed to enable or disable running finalizers on exit. Running finalizers on exit was disabled by default. If enabled, then the finalizers of all objects whose finalizers had not yet been automatically invoked were to be run before the Java runtime exits. That behavior is inherently unsafe. It may result in finalizers being called on live objects while other threads are concurrently manipulating those objects, resulting in erratic behavior or deadlock.*

Throws `UnsupportedOperationException`.

The call `System.runFinalizersOnExit()` is effectively equivalent to the call:

```
Runtime.runFinalizersOnExit()
```

Parameters:

`value - ignored`

Since:

JDK1.1

See Also:

Runtime.runFinalizersOnExit(boolean)

load

```
public static void load(String filename)
```

Loads the native library specified by the filename argument. The filename argument must be an absolute path name. If the filename argument, when stripped of any platform-specific library prefix, path, and file extension, indicates a library whose name is, for example, L, and a native library called L is statically linked with the VM, then the JNI_OnLoad_L function exported by the library is invoked rather than attempting to load a dynamic library. A filename matching the argument does not have to exist in the file system. See the JNI Specification for more details. Otherwise, the filename argument is mapped to a native library image in an implementation-dependent manner.

The call `System.load(name)` is effectively equivalent to the call:

```
Runtime.getRuntime().load(name)
```

Parameters:

`filename` - the file to load.

Throws:

`SecurityException` - if a security manager exists and its checkLink method doesn't allow loading of the specified dynamic library

`UnsatisfiedLinkError` - if either the filename is not an absolute path name, the native library is not statically linked with the VM, or the library cannot be mapped to a native library image by the host system.

`NullPointerException` - if filename is null

See Also:

Runtime.load(java.lang.String), SecurityManager.checkLink(java.lang.String)

loadLibrary

```
public static void loadLibrary(String libname)
```

Loads the native library specified by the `libname` argument. The `libname` argument must not contain any platform specific prefix, file extension or path. If a native library called `libname` is statically linked with the VM, then the JNI_OnLoad_`libname` function exported by the library is invoked. See the

JNI Specification for more details. Otherwise, the libname argument is loaded from a system library location and mapped to a native library image in an implementation- dependent manner.

The call `System.loadLibrary(name)` is effectively equivalent to the call

```
Runtime.getRuntime().loadLibrary(name)
```

Parameters:

`libname - the name of the library.`

Throws:

`SecurityException - if a security manager exists and its checkLink method doesn't allow loading of the specified dynamic library`

`UnsatisfiedLinkError - if either the libname argument contains a file path, the native library is not statically linked with the VM, or the library cannot be mapped to a native library image by the host system.`

`NullPointerException - if libname is null`

See Also:

`Runtime.loadLibrary(java.lang.String)`, `SecurityManager.checkLink(java.lang.String)`

mapLibraryName

```
public static String mapLibraryName(String libname)
```

Maps a library name into a platform-specific string representing a native library.

Parameters:

`libname - the name of the library.`

Returns:

`a platform-dependent native library name.`

Throws:

`NullPointerException - if libname is null`

Since:

`1.2`

See Also:

`loadLibrary(java.lang.String)`, `ClassLoader.findLibrary(java.lang.String)`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.